

Finite-sub-blocklength Latency Minimization for Uplink and Downlink Communications: Theoretical Formulation and Solution Methods

July 6, 2026

Abstract

This report presents the current uplink and downlink work on finite-sub-blocklength latency minimization. It introduces a common notation layer, defines the payload communication setting, derives the finite-sub-blocklength rate expressions used in both links, formulates the uplink and downlink latency-minimization problems, and then develops two solution frameworks: the *Training Only Method* and the *Training + Testing Method*.

1 Introduction

Short-packet communication cannot be analyzed accurately through asymptotic capacity alone. When the codeword length is finite, the achievable rate depends not only on channel quality and transmit power, but also on sub-blocklength and the required decoding reliability. For latency-sensitive multiuser communication, this creates a coupled design problem: the number of transmitted bits, the chosen sub-blocklength, and the precoder must be selected jointly.

The present work studies this problem in both uplink and downlink settings. In the uplink, each user is handled through a user-wise sequence of transmission blocks, and the main difficulty is that the number of required blocks is not known a priori. In the downlink, all active users share the same transmission epoch, so the main difficulty is inter-user interference and the resulting coupling of feasible rates.

The report

1. establish a common mathematical notation for uplink and downlink,
2. define the payload communication setting that determines what each block is asked to deliver,
3. derive the corresponding finite-sub-blocklength uplink and downlink formulations, and
4. present the *Training Only Method* and the *Training + Testing Method* in algorithmic form.

2 System Model and Common Notation

2.1 Core Variables

Table 1 lists the symbols that are shared throughout the report.

The latency of user k is defined by

$$\tau_k \frac{1}{f_{s,k}} \sum_{\ell} n_{k,\ell}, \quad (1)$$

Table 1: Common notation

Symbol	Meaning
$\mathcal{K} = \{1, 2, \dots, K\}$	Set of users.
$k \in \mathcal{K}$	User index.
ℓ	Transmission-block index. In uplink it is per user; in downlink it is per Base Station.
B_k	Total payload of user k in bits.
$b_{k,\ell}^{\text{req}}$	Requested bits to send for user k in block ℓ . In the present report, $b_{k,\ell}^{\text{req}} = B_{k,\text{rem}}^{(\ell)}$.
$b_{k,\ell}$	Bits actually sent to user k in block ℓ .
$B_{k,\text{rem}}^{(\ell)}$	Remaining unsent bits of user k when block ℓ starts.
$n_{k,\ell}$	sub-blocklength assigned to user k in block ℓ , measured in channel uses.
T_k	Maximum admissible sub-blocklength for user k within one coherence-limited transmission opportunity.
$f_{s,k}$	Symbol rate of user k in symbols per second.
τ_k	End-to-end latency of user k .
ε_k	Target packet error probability of user k .
P_k	Uplink power budget of user k .
P_B	Total transmit-power budget of the downlink Base Station Precoder.
d_k	Number of transmitted spatial streams of user k .
$\mathcal{A}_\ell$	Active-user set in downlink block ℓ .
q	One query, i.e. one visited state presented to the learned model.
$\mathcal{Q}$	Collection of training queries used in the Training + Testing Method.

and the network objective is to minimize the aggregate latency

$$= \sum_{k \in \mathcal{K}} \tau_k. \quad (2)$$

2.2 Finite-sub-blocklength Template

For both uplink and downlink, the achievable rate is described through the normal approximation [1]. Once the specific matrix $\mathbf{A}_{k,\ell}^{(m)}$ has been defined, the corresponding capacity term is

$$C_{k,\ell}^{(m)} = \log_2 \det(\mathbf{I} + \mathbf{A}_{k,\ell}^{(m)}), \quad (3)$$

and the channel-dispersion term is

$$V_{k,\ell}^{(m)} = (\log_2 e)^2 \text{tr} \left[\mathbf{A}_{k,\ell}^{(m)} (\mathbf{A}_{k,\ell}^{(m)} + 2\mathbf{I}) (\mathbf{A}_{k,\ell}^{(m)} + \mathbf{I})^{-2} \right]. \quad (4)$$

Therefore, for sub-blocklength $n_{k,\ell}$ and reliability target ε_k , the finite-sub-blocklength rate is

$$R_{k,\ell}^{(m)} = C_{k,\ell}^{(m)} - \sqrt{\frac{V_{k,\ell}^{(m)}}{n_{k,\ell}}} Q^{-1}(\varepsilon_k). \quad (5)$$

If $b_{k,\ell}$ information bits are sent over $n_{k,\ell}$ channel uses, feasibility requires

$$\frac{b_{k,\ell}}{n_{k,\ell}} \leq R_{k,\ell}^{(m)}. \quad (6)$$

Multiplying both sides by $n_{k,\ell}$ yields

$$b_{k,\ell} \leq n_{k,\ell} R_{k,\ell}^{(m)}. \quad (7)$$

Since the service variable is integer-valued, the largest feasible blockwise service is

$$b_{k,\ell}^{\max,(m)} = \left\lfloor n_{k,\ell} R_{k,\ell}^{(m)} \right\rfloor. \quad (8)$$

Equation (8) is central throughout the report: it converts a continuous finite-sub-blocklength rate into an integer blockwise service limit.

3 Communication Scenario

Before separating uplink and downlink, it is useful to define the traffic interpretation used throughout the report. This scenario determines what each block is asked to transmit and how latency is accumulated across blocks.

3.1 Payload Communication

In payload communication, each user begins with a finite payload B_k . The scheduler stops only when this payload is fully drained. The number of bits to send in block ℓ is the current remainder:

$$b_{k,\ell}^{\text{req}} = B_{k,\text{rem}}^{(\ell)}. \quad (9)$$

After the block is committed, the remainder is updated as

$$B_{k,\text{rem}}^{(\ell+1)} = B_{k,\text{rem}}^{(\ell)} - b_{k,\ell}. \quad (10)$$

Thus, payload communication is a workload-draining problem: every decision is made with respect to the residual bits that still have to be delivered.

3.2 Uplink Payload

For uplink payload communication, each user evolves independently according to its own remainder recursion. User k creates a new block only if its current block cannot finish the entire remaining payload. Thus, the block count L_k is an output of the optimization.

3.3 Downlink Payload

For downlink payload communication, the active-user set changes with time:

$$\mathcal{A}_\ell = \{k \in \mathcal{K} : B_{k,\text{rem}}^{(\ell)} > 0\}. \quad (11)$$

As users finish their payloads they leave the active set, which changes both the interference structure and the feasible rate region of the remaining users.

4 Uplink Formulation

4.1 Effective Uplink Signal Model

In the uplink, user k in block ℓ is represented through the effective MIMO relation

$$\mathbf{y}_{k,\ell}^{(\text{UL})} = \mathbf{H}_{k,\ell}^{(\text{UL})} \mathbf{F}_{k,\ell}^{(\text{UL})} \mathbf{s}_{k,\ell} + \mathbf{z}_{k,\ell}^{(\text{UL})}, \quad (12)$$

where

$$\mathbf{H}_{k,\ell}^{(\text{UL})} \in \mathbb{C}^{N_{r,k} \times N_{t,k}}, \quad \mathbf{F}_{k,\ell}^{(\text{UL})} \in \mathbb{C}^{N_{t,k} \times d_k}, \quad (13)$$

$\mathbf{s}_{k,\ell}$ is the transmitted symbol vector with identity covariance, and $\mathbf{z}_{k,\ell}^{(\text{UL})}$ is an AWGN vector. Hence its covariance matrix is

$$\Sigma_{k,\ell}^{(\text{UL})} \sigma_k^2 \mathbf{I}. \quad (14)$$

The desired-signal covariance is

$$\mathbf{S}_{k,\ell}^{(\text{UL})} = \mathbf{H}_{k,\ell}^{(\text{UL})} \mathbf{F}_{k,\ell}^{(\text{UL})} (\mathbf{F}_{k,\ell}^{(\text{UL})})^H (\mathbf{H}_{k,\ell}^{(\text{UL})})^H, \quad (15)$$

and the whitened matrix becomes

$$\mathbf{A}_{k,\ell}^{(\text{UL})} = (\Sigma_{k,\ell}^{(\text{UL})})^{-1/2} \mathbf{S}_{k,\ell}^{(\text{UL})} (\Sigma_{k,\ell}^{(\text{UL})})^{-1/2}. \quad (16)$$

Substituting (16) into (3)–(5) gives $C_{k,\ell}^{(\text{UL})}$, $V_{k,\ell}^{(\text{UL})}$, and $R_{k,\ell}^{(\text{UL})}$.

4.2 Uplink Payload Formulation

In payload communication, the number of blocks required by user k is not known in advance. Let L_k denote the number of blocks eventually used by user k . This terminal block index is induced by the resulting optimization and is therefore an output of the optimization rather than an independent optimization parameter. The uplink payload problem is

$$\min_{\{b_{k,\ell}, n_{k,\ell}, \mathbf{F}_{k,\ell}^{(\text{UL})}\}} \sum_{k \in \mathcal{K}} \frac{1}{f_{s,k}} \sum_{\ell=1}^{L_k} n_{k,\ell} \quad (17a)$$

$$\text{s.t.} \quad \frac{b_{k,\ell}}{n_{k,\ell}} \leq R_{k,\ell}^{(\text{UL})}, \quad \forall k, \ell, \quad (17b)$$

$$\sum_{\ell=1}^{L_k} b_{k,\ell} = B_k, \quad \forall k, \quad (17c)$$

$$n_{k,\ell} \in \{1, \dots, T_k\}, \quad \forall k, \ell, \quad (17d)$$

$$\|\mathbf{F}_{k,\ell}^{(\text{UL})}\|_F^2 \leq P_k, \quad \forall k, \ell. \quad (17e)$$

The remaining-bits recursion is

$$B_{k,\text{rem}}^{(1)} = B_k, \quad B_{k,\text{rem}}^{(\ell+1)} = B_{k,\text{rem}}^{(\ell)} - b_{k,\ell}, \quad (18)$$

and the scheduler terminates once $B_{k,\text{rem}}^{(\ell)} = 0$.

5 Downlink Formulation

5.1 Interference-Coupled Downlink Signal Model

In the downlink, all active users in block ℓ are transmitted simultaneously. The received signal of user k is

$$\mathbf{y}_{k,\ell}^{(\text{DL})} = \mathbf{H}_{k,\ell}^{(\text{DL})} \mathbf{F}_{k,\ell}^{(\text{DL})} \mathbf{s}_{k,\ell} + \sum_{j \in \mathcal{A}_\ell \setminus \{k\}} \mathbf{H}_{k,\ell}^{(\text{DL})} \mathbf{F}_{j,\ell}^{(\text{DL})} \mathbf{s}_{j,\ell} + \mathbf{z}_{k,\ell}^{(\text{DL})}, \quad (19)$$

where

$$\mathbf{H}_{k,\ell}^{(\text{DL})} \in \mathbb{C}^{N_{r,k} \times N_b}, \quad \mathbf{F}_{k,\ell}^{(\text{DL})} \in \mathbb{C}^{N_b \times d_k}, \quad (20)$$

and $\mathbf{z}_{k,\ell}^{(\text{DL})}$ is receiver noise.

The desired-signal covariance of user k is

$$\mathbf{S}_{k,\ell}^{(\text{DL})} = \mathbf{H}_{k,\ell}^{(\text{DL})} \mathbf{F}_{k,\ell}^{(\text{DL})} (\mathbf{F}_{k,\ell}^{(\text{DL})})^H (\mathbf{H}_{k,\ell}^{(\text{DL})})^H. \quad (21)$$

Its interference-plus-noise covariance is

$$\mathbf{\Sigma}_{k,\ell}^{(\text{DL})} = \sigma_k^2 \mathbf{I} + \sum_{j \in \mathcal{A}_\ell \setminus \{k\}} \mathbf{H}_{k,\ell}^{(\text{DL})} \mathbf{F}_{j,\ell}^{(\text{DL})} (\mathbf{F}_{j,\ell}^{(\text{DL})})^H (\mathbf{H}_{k,\ell}^{(\text{DL})})^H. \quad (22)$$

Hence the downlink whitened matrix is

$$\mathbf{A}_{k,\ell}^{(\text{DL})} = (\mathbf{\Sigma}_{k,\ell}^{(\text{DL})})^{-1/2} \mathbf{S}_{k,\ell}^{(\text{DL})} (\mathbf{\Sigma}_{k,\ell}^{(\text{DL})})^{-1/2}, \quad (23)$$

which leads directly to $C_{k,\ell}^{(\text{DL})}$, $V_{k,\ell}^{(\text{DL})}$, and $R_{k,\ell}^{(\text{DL})}$ through (3)–(5).

5.2 Downlink Payload Formulation

In payload communication, the active-user set is induced by the remaining payload:

$$\mathcal{A}_\ell = \left\{ k \in \mathcal{K} : B_{k,\text{rem}}^{(\ell)} > 0 \right\}. \quad (24)$$

The network-level problem is

$$\min_{\{b_{k,\ell}, n_{k,\ell}, \mathbf{F}_{k,\ell}^{(\text{DL})}\}} \sum_{k \in \mathcal{K}} \frac{1}{f_{s,k}} \sum_{\ell} n_{k,\ell} \quad (25a)$$

$$\text{s.t.} \quad \frac{b_{k,\ell}}{n_{k,\ell}} \leq R_{k,\ell}^{(\text{DL})}, \quad \forall k, \ell, \quad (25b)$$

$$\sum_{\ell} b_{k,\ell} = B_k, \quad \forall k, \quad (25c)$$

$$n_{k,\ell} \in \{1, \dots, T_k\}, \quad \forall k, \ell, \quad (25d)$$

$$\sum_{k \in \mathcal{A}_\ell} \left\| \mathbf{F}_{k,\ell}^{(\text{DL})} \right\|_F^2 \leq P_B, \quad \forall \ell. \quad (25e)$$

The same remaining-bits recursion as in (18) applies, but now the block service of one user depends on the simultaneous service decisions of the others.

6 Training Only Method

6.1 Common Primal-Dual Structure

The *Training Only Method* performs optimization directly on the current channel realization. There is no offline learning stage. Instead, each decision block is solved online through a constrained primal-dual update, with $b_{k,\ell}^{\text{req}} = B_{k,\text{rem}}^{(\ell)}$.

For uplink block (k, ℓ) with bits-to-send value $b_{k,\ell}^{\text{req}}$, define the rate constraint residual

$$g_{k,\ell}^{(\text{UL},r)} = \frac{b_{k,\ell}^{\text{req}}}{n_{k,\ell}} - R_{k,\ell}^{(\text{UL})}, \quad (26)$$

and the power residual

$$g_{k,\ell}^{(\text{UL},p)} = \left\| \mathbf{F}_{k,\ell}^{(\text{UL})} \right\|_F^2 - P_k. \quad (27)$$

For the downlink block, define

$$g_{k,\ell}^{(\text{DL},r)} = \frac{b_{k,\ell}^{\text{req}}}{n_{k,\ell}} - R_{k,\ell}^{(\text{DL})}, \quad k \in \mathcal{A}_\ell, \quad (28)$$

and the common block-power residual

$$g_\ell^{(\text{DL},p)} = \sum_{k \in \mathcal{A}_\ell} \left\| \mathbf{F}_{k,\ell}^{(\text{DL})} \right\|_F^2 - P_B. \quad (29)$$

The blockwise Lagrangian is then formed by combining a rate-maximizing primal objective with nonnegative dual penalties. In uplink,

$$\mathcal{L}_{k,\ell}^{(\text{UL})} = -R_{k,\ell}^{(\text{UL})} + \lambda_{k,\ell}^{(\text{UL},r)} [g_{k,\ell}^{(\text{UL},r)}]_+ + \lambda_{k,\ell}^{(\text{UL},p)} [g_{k,\ell}^{(\text{UL},p)}]_+, \quad (30)$$

while in downlink,

$$\mathcal{L}_\ell^{(\text{DL})} = - \sum_{k \in \mathcal{A}_\ell} R_{k,\ell}^{(\text{DL})} + \sum_{k \in \mathcal{A}_\ell} \lambda_{k,\ell}^{(\text{DL},r)} [g_{k,\ell}^{(\text{DL},r)}]_+ + \lambda_\ell^{(\text{DL},p)} [g_\ell^{(\text{DL},p)}]_+. \quad (31)$$

The primal variables are updated to reduce the Lagrangian, and the dual variables are updated to penalize persistent infeasibility. To monitor the solve, the method tracks three KKT residuals.

For uplink block (k, ℓ) ,

$$r_{p,k,\ell}^{(\text{UL})} = \max \left\{ [g_{k,\ell}^{(\text{UL},r)}]_+, [g_{k,\ell}^{(\text{UL},p)}]_+ \right\}, \quad (32)$$

$$r_{c,k,\ell}^{(\text{UL})} = \max \left\{ \lambda_{k,\ell}^{(\text{UL},r)} [g_{k,\ell}^{(\text{UL},r)}]_+, \lambda_{k,\ell}^{(\text{UL},p)} [g_{k,\ell}^{(\text{UL},p)}]_+ \right\}, \quad (33)$$

$$r_{s,k,\ell}^{(\text{UL})} = \frac{\left\| \mathbf{F}_{k,\ell}^{(\text{UL}), (t)} - \mathbf{F}_{k,\ell}^{(\text{UL}), (t-1)} \right\|_F}{\max \left\{ \left\| \mathbf{F}_{k,\ell}^{(\text{UL}), (t-1)} \right\|_F, \delta_0 \right\}}, \quad (34)$$

where $\delta_0 > 0$ is a small safeguard constant.

For downlink block ℓ ,

$$r_{p,\ell}^{(\text{DL})} = \max \left\{ \max_{k \in \mathcal{A}_\ell} [g_{k,\ell}^{(\text{DL},r)}]_+, [g_\ell^{(\text{DL},p)}]_+ \right\}, \quad (35)$$

$$r_{c,\ell}^{(\text{DL})} = \max \left\{ \max_{k \in \mathcal{A}_\ell} \lambda_{k,\ell}^{(\text{DL},r)} [g_{k,\ell}^{(\text{DL},r)}]_+, \lambda_\ell^{(\text{DL},p)} [g_\ell^{(\text{DL},p)}]_+ \right\}, \quad (36)$$

$$r_{s,\ell}^{(\text{DL})} = \max_{k \in \mathcal{A}_\ell} \frac{\left\| \mathbf{F}_{k,\ell}^{(\text{DL}), (t)} - \mathbf{F}_{k,\ell}^{(\text{DL}), (t-1)} \right\|_F}{\max \left\{ \left\| \mathbf{F}_{k,\ell}^{(\text{DL}), (t-1)} \right\|_F, \delta_0 \right\}}, \quad (37)$$

The online solve is declared converged when

$$r_p \leq \epsilon_p, \quad r_c \leq \epsilon_c, \quad r_s \leq \epsilon_s, \quad (38)$$

and it is declared stationary infeasible when

$$r_s \leq \epsilon_s \quad \text{but} \quad r_p > \epsilon_p. \quad (39)$$

6.2 Uplink Training Only Method

The uplink training-only method is fundamentally sequential, but it follows the same *converge first, allocate second, reduce third* logic as the downlink method. For each user-local block, the solver first converges the full-sub-blocklength uplink precoder at $n_{k,\ell} = T_k$, then computes the supported bits, and only then attempts sub-blocklength reduction if the entire payload remainder has already been met. The full-block service is

$$b_{k,\ell}^{(T)} = \min \left\{ B_{k,\text{rem}}^{(\ell)}, \left\lfloor T_k R_{k,\ell}^{(\text{UL})}(T_k) \right\rfloor \right\}. \quad (40)$$

Sub-blocklength reduction is attempted only after the current payload remainder has been fully met, namely when $b_{k,\ell}^{(T)} = B_{k,\text{rem}}^{(\ell)}$. If the current payload remainder is only partially served, the block is committed at $n_{k,\ell} = T_k$ and the method moves to the next request without trying to reduce sub-blocklength.

Algorithm 1 Training-Only Uplink User Solver

Require: User k , payload B_k , tolerances $\epsilon_p, \epsilon_c, \epsilon_s$

Ensure: Block sequence $\{(b_{k,\ell}, n_{k,\ell}, \mathbf{F}_{k,\ell}^{(\text{UL})})\}$

```

1: Set  $B_{k,\text{rem}}^{(1)} \leftarrow B_k$ 
2: Initialize  $\ell \leftarrow 1$ 
3: while  $B_{k,\text{rem}}^{(\ell)} > 0$  do
4:   Set  $b_{k,\ell}^{\text{req}} \leftarrow B_{k,\text{rem}}^{(\ell)}$ 
5:   Set  $n_{k,\ell} \leftarrow T_k$  and initialize the primal-dual state  $(\mathbf{F}_{k,\ell}^{(\text{UL})}, \lambda_{k,\ell}^{(\text{UL},r)}, \lambda_{k,\ell}^{(\text{UL},p)})$ 
6:   repeat
7:     Update  $\mathbf{F}_{k,\ell}^{(\text{UL})}$  using the uplink Lagrangian (30)
8:     Update  $\lambda_{k,\ell}^{(\text{UL},r)}$  and  $\lambda_{k,\ell}^{(\text{UL},p)}$ 
9:     Measure  $r_{p,k,\ell}^{(\text{UL})}$ ,  $r_{c,k,\ell}^{(\text{UL})}$ , and  $r_{s,k,\ell}^{(\text{UL})}$  from (32)–(34)
10:    until the uplink solve satisfies (38) or (39)
11:    Restore the best accepted state: feasible if available, otherwise minimum-violation
12:    Compute  $b_{k,\ell}^{(T)} = \min\{b_{k,\ell}^{\text{req}}, \lfloor T_k R_{k,\ell}^{(\text{UL})}(T_k) \rfloor\}$ 
13:    if  $b_{k,\ell}^{(T)} = 0$  then
14:      Set  $b_{k,\ell} \leftarrow 0$  and keep  $n_{k,\ell} = T_k$ 
15:      Declare the remaining payload infeasible under the current state and stop
16:    else if  $b_{k,\ell}^{(T)} < b_{k,\ell}^{\text{req}}$  then
17:      Commit  $b_{k,\ell} = b_{k,\ell}^{(T)}$  and set  $n_{k,\ell} = T_k$ 
18:    else
19:      Set  $b_{k,\ell} = b_{k,\ell}^{\text{req}}$ 
20:      Store the current accepted state
21:      while a smaller candidate sub-blocklength exists do
22:        Tentatively replace  $n_{k,\ell}$  by the next smaller candidate
23:        Re-solve the candidate state with the same primal-dual loop above
24:        Restore the best candidate state: feasible if available, otherwise the previous accepted
state
25:        if  $\frac{b_{k,\ell}}{n_{k,\ell}} \leq R_{k,\ell}^{(\text{UL})}$  and  $\|\mathbf{F}_{k,\ell}^{(\text{UL})}\|_F^2 \leq P_k$  then
26:          Accept the candidate and store the updated accepted state
27:        else
28:          Stop reduction and keep the previous accepted state
29:        Commit the smallest accepted  $n_{k,\ell}$  and corresponding precoder state
30:      Update  $B_{k,\text{rem}}^{(\ell+1)} \leftarrow B_{k,\text{rem}}^{(\ell)} - b_{k,\ell}$ 
31:       $\ell \leftarrow \ell + 1$ 

```

Algorithm 1 solves the uplink payload problem by updating the remaining workload after every committed block.

6.3 Downlink Training Only Method

In the downlink, a whole active-user block must be solved jointly. The same three-phase logic is applied, but now on a coupled active-user set. The method first converges the joint full-sub-blocklength precoder at the coherence limits $\{T_k\}_{k \in \mathcal{A}_\ell}$, then computes the supported bits of all requested users, removes users with zero supported service, and only then reduces sub-blocklength for the users whose full payload remainders are already supported. This produces

the full-block supported service

$$b_{k,\ell}^{(T)} = \min \left\{ b_{k,\ell}^{\text{req}}, \left\lfloor T_k R_{k,\ell}^{(\text{DL})}(T_k) \right\rfloor \right\}, \quad k \in \mathcal{A}_\ell. \quad (41)$$

Users with $b_{k,\ell}^{(T)} = 0$ are removed from transmission in that block. For users that satisfy $b_{k,\ell}^{(T)} = b_{k,\ell}^{\text{req}}$, the method tries to reduce sub-blocklength while keeping the already committed bits fixed. Since the block is coupled, a candidate reduction may invalidate another user's committed service. The method therefore re-optimizes the affected active-user subset whenever necessary and accepts a candidate reduction only if all committed bits remain feasible.

Algorithm 2 Training-Only Downlink Block Solver

Require: Active-user set $\mathcal{A}_\ell$, remaining payloads $\{B_{k,\text{rem}}^{(\ell)}\}_{k \in \mathcal{A}_\ell}$, tolerances $\epsilon_p, \epsilon_c, \epsilon_s$

Ensure: Block plan $\{(b_{k,\ell}, n_{k,\ell}, \mathbf{F}_{k,\ell}^{(\text{DL})})\}_{k \in \mathcal{A}_\ell}$

- 1: Store the requested-user set as $\mathcal{A}_\ell^{\text{req}} \leftarrow \mathcal{A}_\ell$
- 2: Set the current transmit set $\mathcal{A}_\ell^{\text{tx}} \leftarrow \mathcal{A}_\ell^{\text{req}}$
- 3: **for** each $k \in \mathcal{A}_\ell^{\text{req}}$ **do**
- 4: Set $b_{k,\ell}^{\text{req}} \leftarrow B_{k,\text{rem}}^{(\ell)}$
- 5: Set $n_{k,\ell} \leftarrow T_k$
- 6: Initialize the active-user precoders and multipliers on $\mathcal{A}_\ell^{\text{tx}}$
- 7: **repeat**
- 8: Update the active-user precoders using the block Lagrangian (31)
- 9: Update the multipliers $\{\lambda_{k,\ell}^{(\text{DL},r)}\}$ and $\lambda_\ell^{(\text{DL},p)}$
- 10: Measure $r_{p,\ell}^{(\text{DL})}$, $r_{c,\ell}^{(\text{DL})}$, and $r_{s,\ell}^{(\text{DL})}$ from (35)–(37)
- 11: **until** the downlink solve satisfies (38) or (39)
- 12: Restore the best accepted block state: feasible if available, otherwise minimum-violation
- 13: **repeat**
- 14: **for** each requested user $k \in \mathcal{A}_\ell^{\text{req}}$ **do**
- 15: **if** $k \in \mathcal{A}_\ell^{\text{tx}}$ **then**
- 16: Set $b_{k,\ell} \leftarrow \min\{b_{k,\ell}^{\text{req}}, \lfloor n_{k,\ell} R_{k,\ell}^{(\text{DL})}(\mathbf{n}_\ell) \rfloor\}$
- 17: **else**
- 18: Set $b_{k,\ell} \leftarrow 0$
- 19: **if** every user in $\mathcal{A}_\ell^{\text{tx}}$ satisfies $b_{k,\ell} > 0$ **then**
- 20: Stop pruning zero-service users
- 21: **else**
- 22: Remove from $\mathcal{A}_\ell^{\text{tx}}$ every user with $b_{k,\ell} = 0$
- 23: **if** $\mathcal{A}_\ell^{\text{tx}} \neq \emptyset$ **then**
- 24: Re-solve the reduced active-user block with the same primal-dual loop above
- 25: Restore the best accepted reduced-set state: feasible if available, otherwise minimum-violation
- 26: **until** every remaining active user receives positive full-state service or $\mathcal{A}_\ell^{\text{tx}} = \emptyset$
- 27: **for** each user $k \in \mathcal{A}_\ell^{\text{tx}}$ satisfying $b_{k,\ell} = b_{k,\ell}^{\text{req}}$ **do**
- 28: Store the current accepted block state
- 29: **while** a smaller candidate sub-blocklength exists for user k **do**
- 30: Tentatively decrease $n_{k,\ell}$ while holding the other active-user sub-blocklengths fixed
- 31: Re-optimize the affected active-user block with the same primal-dual loop above
- 32: Restore the best candidate block state: feasible if available, otherwise the previous accepted state
- 33: **if** $\frac{b_{j,\ell}}{n_{j,\ell}} \leq R_{j,\ell}^{(\text{DL})}$ for all $j \in \mathcal{A}_\ell^{\text{tx}}$ and $\sum_{j \in \mathcal{A}_\ell^{\text{tx}}} \|\mathbf{F}_{j,\ell}^{(\text{DL})}\|_F^2 \leq P_B$ **then**
- 34: Accept the candidate and store the updated accepted state
- 35: **else**
- 36: Stop reducing user k
- 37: **for** each requested user $k \in \mathcal{A}_\ell^{\text{req}}$ **do**
- 38: Update $B_{k,\text{rem}}^{(\ell+1)} \leftarrow B_{k,\text{rem}}^{(\ell)} - b_{k,\ell}$
- 39: Commit the final joint block plan

The training-only downlink method is therefore a *converge first, allocate second, reduce third* procedure. This ordering is essential because the finite-sub-blocklength rate of each user depends on the full active-user beam set.

7 Training + Testing Method

7.1 Method Principle

The *Training + Testing Method* separates optimization into two stages. During training, a pre-coder mapping is learned from a collection of queries produced by the same finite-sub-blocklength scheduling logic that will later be used at test time. During testing, the online optimization step is replaced by model inference, followed by the same feasibility checks and the same sub-blocklength-reduction logic.

The central training object is a query. In general form, a query $q \in \mathcal{Q}$ contains

$$q = (\text{channel state, active-user state, candidate sub-blocklength state, bits-to-send state, covariance state, reliability target}). \quad (42)$$

Thus, a query is simply one scheduling state at which the model is asked to predict a feasible finite-sub-blocklength transmission strategy. In the present formulation, the training loss uses *uniform-per-query* averaging, so every stored query contributes equally to the empirical objective.

7.2 Uplink Training + Testing Method

7.2.1 Training stage

For uplink user k , a query at block ℓ has the form

$$q_{k,\ell}^{(\text{UL})} = (\mathbf{H}_{k,\ell}^{(\text{UL})}, \boldsymbol{\Sigma}_{k,\ell}^{(\text{UL})}, n, b_{k,\ell}^{\text{req}}, \varepsilon_k), \quad (43)$$

where n is the candidate sub-blocklength being queried and $b_{k,\ell}^{\text{req}}$ is the number of bits to send in that state. The query set $\mathcal{Q}_k^{(\text{UL})}$ is built by running the same causal block logic used by the scheduler itself and recording every visited candidate state. In payload communication, the number of bits to send is $b_{k,\ell}^{\text{req}} = B_{k,\text{rem}}^{(\ell)}$.

Let θ_k denote the trainable parameters of the uplink model for user k . The uniform-per-query training objective is

$$\mathcal{J}_k^{(\text{UL})}(\theta_k) = \frac{1}{|\mathcal{Q}_k^{(\text{UL})}|} \sum_{q \in \mathcal{Q}_k^{(\text{UL})}} \left(-R(q; \theta_k) + \lambda_k^{(r)} \left[\frac{b^{\text{req}}(q)}{n(q)} - R(q; \theta_k) \right]_+ + \lambda_k^{(p)} [P(q, \theta_k) - P_k]_+ \right), \quad (44)$$

where $R(q; \theta_k)$ is the finite-sub-blocklength rate induced by the predicted uplink precoder, $n(q)$ is the queried sub-blocklength, $b^{\text{req}}(q)$ is the bits-to-send value encoded in query q , and $P(q, \theta_k)$ is the transmit power induced by the predicted precoder. The coefficient $\lambda_k^{(r)}$ penalizes rate shortfall relative to the bits-to-send value, whereas $\lambda_k^{(p)}$ penalizes power-budget violation.

Algorithm 3 Uplink Training Stage

Require: Training episodes for uplink user k

Ensure: Query set $\mathcal{Q}_k^{(\text{UL})}$ and trained parameters θ_k

```
1: Initialize  $\mathcal{Q}_k^{(\text{UL})} \leftarrow \emptyset$ 
2: for each training episode do
3:   Initialize the episode remainder state from the episode payload
4:   for each visited block  $\ell$  of user  $k$  do
5:     Set  $b_{k,\ell}^{\text{req}} \leftarrow B_{k,\text{rem}}^{(\ell)}$ 
6:     Set the first candidate sub-blocklength to  $n \leftarrow T_k$ 
7:     while  $n$  is under consideration do
8:       Form the query  $q_{k,\ell}^{(\text{UL})}$  in (43) and add it to  $\mathcal{Q}_k^{(\text{UL})}$ 
9:       Evaluate the current state and compute

$$b_{k,\ell}^{(n)} = \min \left\{ b_{k,\ell}^{\text{req}}, \left\lfloor n R_{k,\ell}^{(\text{UL})}(n) \right\rfloor \right\}$$

10:      if  $b_{k,\ell}^{(n)} < b_{k,\ell}^{\text{req}}$  then
11:        Stop collecting additional queries for this sub-block
12:      else
13:        Replace  $n$  by the next smaller candidate sub-blocklength
14:      Advance the episode remainder by the committed service
15: Train  $\theta_k$  by minimizing (44)
```

7.2.2 Testing stage

At test time, the trained model replaces the online uplink solve. The block is first examined at the full available sub-blocklength. If the full block does not support the complete request, the scheduler commits only the supported service at $n = T_k$. If the full block already supports the entire request, the sub-blocklength is reduced and the model is queried again at smaller candidate sub-blocklengths until the smallest feasible one is identified.

Algorithm 4 Uplink Testing Stage

Require: Trained uplink model θ_k , user k , payload B_k **Ensure:** Final uplink block plan

- 1: Set $B_{k,\text{rem}}^{(1)} \leftarrow B_k$
 - 2: **for** each visited block ℓ **do**
 - 3: Set $b_{k,\ell}^{\text{req}} \leftarrow B_{k,\text{rem}}^{(\ell)}$
 - 4: Infer the uplink precoder at $n = T_k$
 - 5: Compute the full-block supported service

$$b_{k,\ell}^{(T)} = \min \left\{ b_{k,\ell}^{\text{req}}, \left\lfloor T_k R_{k,\ell}^{(\text{UL})}(T_k) \right\rfloor \right\}$$
 - 6: **if** $b_{k,\ell}^{(T)} < b_{k,\ell}^{\text{req}}$ **then**
 - 7: Commit $n_{k,\ell} = T_k$ and $b_{k,\ell} = b_{k,\ell}^{(T)}$
 - 8: **else**
 - 9: Set $b_{k,\ell} = b_{k,\ell}^{\text{req}}$
 - 10: Set $n_{k,\ell} \leftarrow T_k$
 - 11: Store the current accepted state
 - 12: **while** a smaller candidate sub-blocklength exists **do**
 - 13: Tentatively replace $n_{k,\ell}$ by the next smaller candidate
 - 14: Infer the uplink precoder for the updated query
 - 15: **if** $\frac{b_{k,\ell}}{n_{k,\ell}} \leq R_{k,\ell}^{(\text{UL})}(n_{k,\ell})$ and $P(q_{k,\ell}^{(\text{UL})}, \theta_k) \leq P_k$ **then**
 - 16: Accept the reduction and store the updated accepted state
 - 17: **else**
 - 18: Stop reduction and keep the previous accepted state
 - 19: Commit the smallest accepted $n_{k,\ell}$ and corresponding precoder
 - 20: Update $B_{k,\text{rem}}^{(\ell+1)} \leftarrow B_{k,\text{rem}}^{(\ell)} - b_{k,\ell}$
-

7.3 Downlink Training + Testing Method

7.3.1 Training stage

The downlink training state is joint across the active users of a block. A query for block ℓ can be written as

$$q_\ell^{(\text{DL})} = (\{\mathbf{H}_{k,\ell}^{(\text{DL})}\}_{k \in \mathcal{A}_\ell}, \mathcal{A}_\ell, \{n_{k,\ell}\}_{k \in \mathcal{A}_\ell}, \{b_{k,\ell}^{\text{req}}\}_{k \in \mathcal{A}_\ell}, \{\Sigma_{k,\ell}^{(\text{DL})}\}_{k \in \mathcal{A}_\ell}, \{\varepsilon_k\}_{k \in \mathcal{A}_\ell}), \quad (45)$$

where $\mathcal{A}_\ell$ is the active-user set of the block. The query set $\mathcal{Q}^{(\text{DL})}$ is built causally by running the same joint scheduling logic used at test time and recording every visited joint state. In payload communication, the number of bits to send is $b_{k,\ell}^{\text{req}} = B_{k,\text{rem}}^{(\ell)}$.

Let $\theta = \{\theta_k\}_{k=1}^K$ denote the trainable parameters of the downlink model collection. The uniform-per-query training objective is

$$\begin{aligned} \mathcal{J}^{(\text{DL})}(\theta) = \frac{1}{|\mathcal{Q}^{(\text{DL})}|} \sum_{q \in \mathcal{Q}^{(\text{DL})}} \bigg(& - \sum_{k \in \mathcal{A}(q)} R_k(q; \theta) \\ & + \sum_{k \in \mathcal{A}(q)} \lambda_k^{(r)} \left[\frac{b_k^{\text{req}}(q)}{n_k(q)} - R_k(q; \theta) \right]_+ \\ & + \lambda^{(p)} \left[\sum_{k \in \mathcal{A}(q)} P_k(q, \theta) - P_B \right]_+ \bigg). \end{aligned} \quad (46)$$

where $\mathcal{A}(q)$ is the active-user set encoded in query q , $n_k(q)$ is the sub-blocklength associated with active user k in that query, $b_k^{\text{req}}(q)$ is the number of bits to send for active user k , and

$P_k(q, \theta)$ is the transmit power induced for active user k by the predicted downlink precoder. The penalty coefficients $\lambda_k^{(r)}$ and $\lambda^{(p)}$ enforce the rate constraints and the total base-station power constraint, respectively.

Algorithm 5 Downlink Training Stage

Require: Training episodes

Ensure: Query set $\mathcal{Q}^{(\text{DL})}$ and trained parameters θ

```

1: Initialize  $\mathcal{Q}^{(\text{DL})} \leftarrow \emptyset$ 
2: for each training episode do
3:   for each visited communication block  $\ell$  do
4:     Determine the requested-user set  $\mathcal{A}_\ell^{\text{req}}$ 
5:     Set the current transmit set  $\mathcal{A}_\ell^{\text{tx}} \leftarrow \mathcal{A}_\ell^{\text{req}}$ 
6:     for each  $k \in \mathcal{A}_\ell^{\text{req}}$  do
7:       Set  $b_{k,\ell}^{\text{req}} \leftarrow B_{k,\text{rem}}^{(\ell)}$ 
8:       Initialize  $n_{k,\ell} \leftarrow T_k$ 
9:       while the current joint sub-blocklength state is under consideration do
10:        Form the joint query  $q_\ell^{(\text{DL})}$  in (45) and add it to  $\mathcal{Q}^{(\text{DL})}$ 
11:        for each  $k \in \mathcal{A}_\ell^{\text{req}}$  do
12:          if  $k \in \mathcal{A}_\ell^{\text{tx}}$  then
13:            Set  $b_{k,\ell} \leftarrow \min \left\{ b_{k,\ell}^{\text{req}}, \left\lfloor n_{k,\ell} R_{k,\ell}^{(\text{DL})}(\mathbf{n}_\ell) \right\rfloor \right\}$ 
14:          else
15:            Set  $b_{k,\ell} \leftarrow 0$ 
16:          if some user in  $\mathcal{A}_\ell^{\text{tx}}$  satisfies  $b_{k,\ell} = 0$  then
17:            Remove from  $\mathcal{A}_\ell^{\text{tx}}$  every user with  $b_{k,\ell} = 0$ 
18:          else if every user in  $\mathcal{A}_\ell^{\text{tx}}$  satisfies  $b_{k,\ell} < b_{k,\ell}^{\text{req}}$  then
19:            Stop collecting additional queries for this block
20:          else
21:            Decrease the candidate sub-blocklengths of users satisfying  $b_{k,\ell} = b_{k,\ell}^{\text{req}}$ 
22:        Advance the episode remainders by the committed service
23: Train  $\theta$  by minimizing (46)

```

7.3.2 Testing stage

At test time, the learned downlink model is queried on the current active-user state of the block. The full joint sub-blocklength vector is evaluated first. Users with zero supported service are removed from the transmit set, and joint sub-blocklength reduction is attempted only for users whose full bits-to-send value is already supported at the current state.

Algorithm 6 Downlink Testing Stage

Require: Trained downlink model θ , remaining payloads $\{B_{k,\text{rem}}^{(\ell)}\}$

Ensure: Final downlink block plan

```
1: for each communication block  $\ell$  do
2:   Determine the requested-user set  $\mathcal{A}_\ell^{\text{req}}$ 
3:   Set the current transmit set  $\mathcal{A}_\ell^{\text{tx}} \leftarrow \mathcal{A}_\ell^{\text{req}}$ 
4:   for each  $k \in \mathcal{A}_\ell^{\text{req}}$  do
5:     Set  $b_{k,\ell}^{\text{req}} \leftarrow B_{k,\text{rem}}^{(\ell)}$ 
6:     Initialize  $n_{k,\ell} \leftarrow T_k$ 
7:   Infer the joint downlink precoders at the full sub-blocklength vector on  $\mathcal{A}_\ell^{\text{tx}}$ 
8:   repeat
9:     for each  $k \in \mathcal{A}_\ell^{\text{req}}$  do
10:      if  $k \in \mathcal{A}_\ell^{\text{tx}}$  then
11:        Set  $b_{k,\ell} \leftarrow \min\left\{b_{k,\ell}^{\text{req}}, \left\lfloor n_{k,\ell} R_{k,\ell}^{(\text{DL})}(\mathbf{n}_\ell) \right\rfloor\right\}$ 
12:      else
13:        Set  $b_{k,\ell} \leftarrow 0$ 
14:      if some user in  $\mathcal{A}_\ell^{\text{tx}}$  satisfies  $b_{k,\ell} = 0$  then
15:        Remove from  $\mathcal{A}_\ell^{\text{tx}}$  every user with  $b_{k,\ell} = 0$ 
16:      if  $\mathcal{A}_\ell^{\text{tx}} \neq \emptyset$  then
17:        Re-infer the joint downlink precoders on the reduced transmit set
18:      else
19:        Stop pruning zero-service users
20:    until every remaining active user receives positive full-state service or  $\mathcal{A}_\ell^{\text{tx}} = \emptyset$ 
21:    for each user  $k \in \mathcal{A}_\ell^{\text{tx}}$  satisfying  $b_{k,\ell} = b_{k,\ell}^{\text{req}}$  do
22:      Store the current accepted state
23:      while a smaller candidate sub-blocklength exists for that user do
24:        Tentatively decrease  $n_{k,\ell}$  while holding the other active-user sub-blocklengths fixed
25:        Infer the joint downlink precoders for the updated state
26:        if  $\frac{b_{j,\ell}}{n_{j,\ell}} \leq R_{j,\ell}^{(\text{DL})}(\mathbf{n}_\ell)$  for all  $j \in \mathcal{A}_\ell^{\text{tx}}$  and  $\sum_{j \in \mathcal{A}_\ell^{\text{tx}}} P_j(q_\ell^{(\text{DL})}, \theta) \leq P_B$  then
27:          Accept the reduction and store the updated accepted state
28:        else
29:          Stop reducing user  $k$  and keep the previous accepted state
30:    for each requested user  $k \in \mathcal{A}_\ell^{\text{req}}$  do
31:      Update  $B_{k,\text{rem}}^{(\ell+1)} \leftarrow B_{k,\text{rem}}^{(\ell)} - b_{k,\ell}$ 
32:  Commit the final active-user set, sub-blocklengths, and transmitted bits of the block
```

8 Conclusion

The report has recast the current work into a theory-centered structure. The development proceeds from a common finite-sub-blocklength model and payload communication setting, to separate uplink and downlink formulations, and then to two solution frameworks.

The uplink problem is characterized by user-wise sequential block creation under finite-sub-blocklength feasibility. The downlink problem is characterized by a jointly coupled interference structure that requires block-level coordination before blockwise service can be committed.

The *Training Only Method* solves each decision state online through constrained primal-dual optimization and feasibility-guided sub-blocklength reduction. The *Training + Testing Method* replaces online optimization by learned precoder inference, but retains the same finite-sub-blocklength service logic through query-based offline training, feasibility checks, and causal sub-blocklength reduction.

Taken together, these formulations provide a complete theoretical description of the current uplink and downlink methodology without relying on implementation details or numerical

experiment summaries.

References

- [1] Y. Polyanskiy, H. V. Poor, and S. Verdú, “Channel coding rate in the finite sub-blocklength regime,” *IEEE Transactions on Information Theory*, vol. 56, no. 5, pp. 2307–2359, 2010.